



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 12

Control de LED RGB mediante señales PWM

Tres potenciómetros, tres canales, colores infinitos

Alumno: Cesar Gael Mancilla Estrada
Clave: 382031
Carrera: Ingeniería en Mecatrónica
Materia: Microcontroladores
Profesor: Ing. Jesús Padrón
Fecha: Marzo de 2025

.^{El} PWM convierte señales analógicas en control digital preciso"

Índice

1. Objetivo	2
2. Introducción	2
2.1. Contexto	2
2.2. Importancia	2
3. Desarrollo	2
3.1. Diagrama en Protoboard	2
3.2. Conexiones del sistema	3
3.3. CMakeLists.txt	3
4. Código C para la Pico W	3
5. Código Python para Visualización	6
6. Preguntas de Repaso	11
7. Conclusiones	12
7.1. Aprendizajes de la práctica	12
7.2. Posibles mejoras	12
8. Referencias	12

1. Objetivo

Aprender el funcionamiento de las señales PWM del microcontrolador como método de control en dispositivos como los LEDs RGB, mediante la lectura de tres potenciómetros que controlan independientemente los canales rojo, verde y azul.

2. Introducción

2.1. Contexto

La modulación por ancho de pulso (PWM) es una técnica fundamental en sistemas embebidos que permite controlar la potencia entregada a una carga sin necesidad de convertidores analógicos. En esta práctica se utiliza PWM para controlar la intensidad de cada color de un LED RGB, logrando una amplia gama de colores a partir de tres señales de control.

2.2. Importancia

El control de LEDs RGB mediante PWM tiene aplicaciones en iluminación arquitectónica, pantallas LED, señalización, retroiluminación de monitores y efectos especiales.

3. Desarrollo

3.1. Diagrama en Protoboard

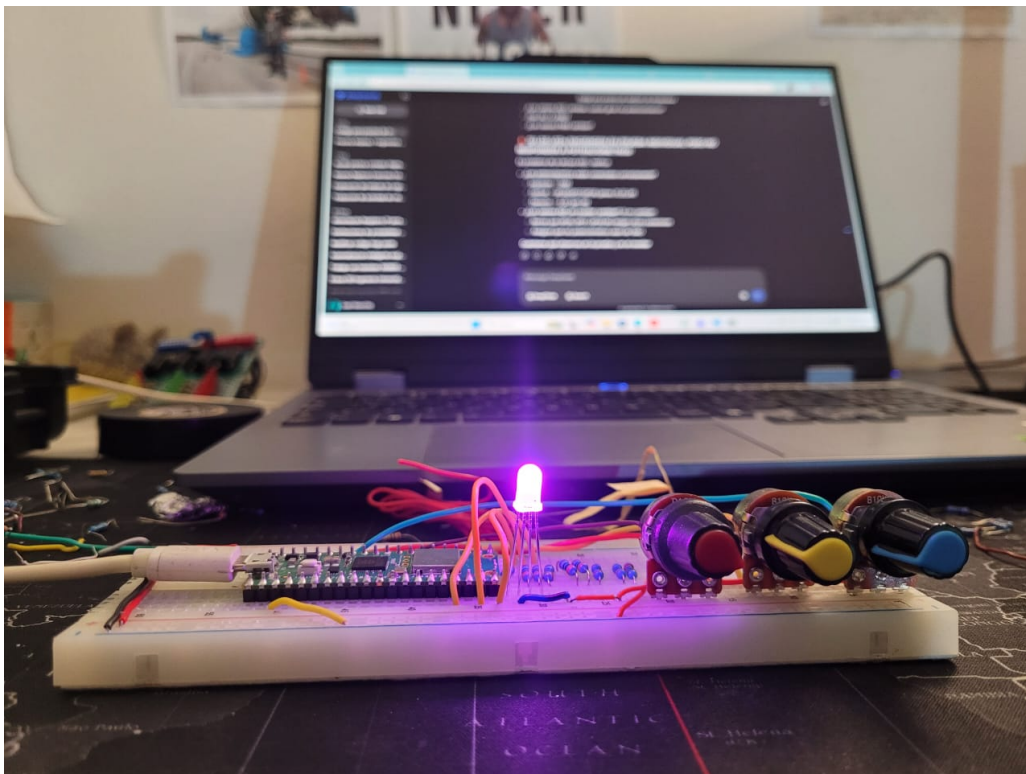


Figura 1: Conexión de potenciómetros y LED RGB en protoboard

3.2. Conexiones del sistema

Tabla 1: Conexiones de potenciómetros y LED RGB

Componente	Pin Pico	Función
Potenciómetro Rojo (centro)	GP26 (ADC0)	Control canal R
Potenciómetro Verde (centro)	GP27 (ADC1)	Control canal G
Potenciómetro Azul (centro)	GP28 (ADC2)	Control canal B
LED RGB - Cátodo/Ánodo	GND / 3.3V	Alimentación
LED RGB - Rojo	GP15 (PWM)	PWM canal R
LED RGB - Verde	GP14 (PWM)	PWM canal G
LED RGB - Azul	GP13 (PWM)	PWM canal B

3.3. CMakeLists.txt

Listing 1: Archivo CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4
5 set(PICO_BOARD pico_w)
6 set(PICO_STDIO_USB 1)
7
8 project(Practica_12)
9
10 pico_sdk_init()
11
12 add_executable(rgb_pwm
13     rgb_pwm.c
14 )
15
16 target_link_libraries(rgb_pwm
17     pico_stdlib
18     hardware_adc
19     hardware_pwm
20     hardware_gpio
21 )
22
23 pico_enable_stdio_usb(rgb_pwm 1)
24 pico_enable_stdio_uart(rgb_pwm 0)
25
26 pico_add_extra_outputs(rgb_pwm)

```

4. Código C para la Pico W

Listing 2: rgb_pwm.c - Control de LED RGB con PWM

```
1  /*
2  * Practica 12: Control de LED RGB mediante PWM
3  * Tres potenciómetros controlan los canales R, G, B
4  */
5
6  #include <stdio.h>
7  #include "pico/stdlib.h"
8  #include "hardware/adc.h"
9  #include "hardware/pwm.h"
10 #include "hardware/gpio.h"
11
12 #define POT_R_PIN 26 // ADC0 - Rojo
13 #define POT_G_PIN 27 // ADC1 - Verde
14 #define POT_B_PIN 28 // ADC2 - Azul
15
16 #define LED_R_PIN 15 // PWM - Rojo
17 #define LED_G_PIN 14 // PWM - Verde
18 #define LED_B_PIN 13 // PWM - Azul
19
20 #define CATHODE_COMMON false
21 #define PWM_WRAP 255
22 #define ADC_MAX 4095
23
24 typedef struct {
25     uint8_t pin_adc;
26     uint8_t pin_pwm;
27     uint16_t raw_value;
28     uint8_t porcentaje;
29     uint8_t pwm_actual;
30 } CanalRGB;
31
32 CanalRGB rojo = {.pin_adc = POT_R_PIN, .pin_pwm = LED_R_PIN};
33 CanalRGB verde = {.pin_adc = POT_G_PIN, .pin_pwm = LED_G_PIN};
34 CanalRGB azul = {.pin_adc = POT_B_PIN, .pin_pwm = LED_B_PIN};
35
36 void init_adc(void);
37 void init_pwm(void);
38 uint16_t leer_adc(uint8_t pin);
39 uint8_t adc_a_pwm(uint16_t adc_val);
40 void actualizar_led(CanalRGB *canal);
41 void imprimir_estado(void);
42
43 int main() {
44     stdio_init_all();
45     sleep_ms(3000);
46
47     printf("Canales de color:\n");
48
49     init_adc();
50     init_pwm();
51 }
```

```
52     uint32_t ultima_impression = 0;
53
54     while (true) {
55         rojo.raw_value = leer_adc(rojo.pin_adc);
56         rojo.porcentaje = (rojo.raw_value * 100) / ADC_MAX;
57
58         verde.raw_value = leer_adc(verde.pin_adc);
59         verde.porcentaje = (verde.raw_value * 100) / ADC_MAX;
60
61         azul.raw_value = leer_adc(azul.pin_adc);
62         azul.porcentaje = (azul.raw_value * 100) / ADC_MAX;
63
64         actualizar_led(&rojo);
65         actualizar_led(&verde);
66         actualizar_led(&azul);
67
68         uint32_t ahora = to_ms_since_boot(get_absolute_time());
69         if (ahora - ultima_impression >= 500) {
70             imprimir_estado();
71             ultima_impression = ahora;
72         }
73
74         sleep_ms(10);
75     }
76 }
77
78 void init_adc(void) {
79     adc_init();
80     adc_gpio_init(rojo.pin_adc);
81     adc_gpio_init(verde.pin_adc);
82     adc_gpio_init(azul.pin_adc);
83 }
84
85 void init_pwm(void) {
86     gpio_set_function(rojo.pin_pwm, GPIO_FUNC_PWM);
87     gpio_set_function(verde.pin_pwm, GPIO_FUNC_PWM);
88     gpio_set_function(azul.pin_pwm, GPIO_FUNC_PWM);
89
90     uint slice_r = pwm_gpio_to_slice_num(rojo.pin_pwm);
91     uint slice_g = pwm_gpio_to_slice_num(verde.pin_pwm);
92     uint slice_b = pwm_gpio_to_slice_num(azul.pin_pwm);
93
94     pwm_set_wrap(slice_r, PWM_WRAP);
95     pwm_set_wrap(slice_g, PWM_WRAP);
96     pwm_set_wrap(slice_b, PWM_WRAP);
97
98     pwm_set_enabled(slice_r, true);
99     pwm_set_enabled(slice_g, true);
100    pwm_set_enabled(slice_b, true);
101
102    pwm_set_gpio_level(rojo.pin_pwm, PWM_WRAP);
```

```

103     pwm_set_gpio_level(verde.pin_pwm, PWM_WRAP);
104     pwm_set_gpio_level(azul.pin_pwm, PWM_WRAP);
105 }
106
107 uint16_t leer_adc(uint8_t pin) {
108     if (pin == 26) adc_select_input(0);
109     else if (pin == 27) adc_select_input(1);
110     else if (pin == 28) adc_select_input(2);
111     else return 0;
112
113     sleep_us(10);
114     return adc_read();
115 }
116
117 uint8_t adc_a_pwm(uint16_t adc_val) {
118     uint8_t pwm = (adc_val * PWM_WRAP) / ADC_MAX;
119
120     if (!CATHODE_COMMON) {
121         pwm = PWM_WRAP - pwm;
122     }
123
124     return pwm;
125 }
126
127 void actualizar_led(CanalRGB *canal) {
128     uint8_t nuevo_pwm = adc_a_pwm(canal->raw_value);
129
130     if (canal->pwm_actual != nuevo_pwm) {
131         canal->pwm_actual = nuevo_pwm;
132         pwm_set_gpio_level(canal->pin_pwm, canal->pwm_actual);
133     }
134 }
135
136 void imprimir_estado(void) {
137     printf("R:%3d%% | G:%3d%% | B:%3d%%\n",
138         rojo.porcentaje, verde.porcentaje, azul.porcentaje);
139     fflush(stdout);
140 }

```

5. Código Python para Visualización

Listing 3: grafica_rgb.py - Visualización en tiempo real

```

1 """
2 Practica 12: Visualizacion grafica del control RGB
3 Muestra los valores de los potenciometros en tiempo real
4 """
5
6 import tkinter as tk
7 from tkinter import ttk

```

```
8 import serial
9 import threading
10 import time
11 import re
12
13 class ControlRGB:
14     def __init__(self, puerto='COM3', baudrate=115200):
15         self.puerto = puerto
16         self.baudrate = baudrate
17         self.serial_activo = False
18         self.ser = None
19
20         self.r_val = 0
21         self.g_val = 0
22         self.b_val = 0
23
24         self.patron = re.compile(r'R:(\d+)% \|| G:(\d+)% \|| B:(\d+)%')
25
26         self.root = tk.Tk()
27         self.root.title("Control RGB - Practica 12")
28         self.root.geometry("800x500")
29         self.root.configure(bg='#2b2b2b')
30
31         self.crear_interfaz()
32
33     def crear_interfaz(self):
34         titulo = tk.Label(self.root, text="Control de LED RGB",
35                           font=('Arial', 24, 'bold'),
36                           bg='#2b2b2b', fg='white')
37         titulo.pack(pady=20)
38
39         frame_conexion = tk.Frame(self.root, bg='#2b2b2b')
40         frame_conexion.pack(pady=10)
41
42         tk.Label(frame_conexion, text="Puerto:",
43                 bg='#2b2b2b', fg='white').pack(side=tk.LEFT, padx=5)
44
45         self.entry_puerto = tk.Entry(frame_conexion, width=10)
46         self.entry_puerto.pack(side=tk.LEFT, padx=5)
47         self.entry_puerto.insert(0, self.puerto)
48
49         self.btn_conectar = tk.Button(frame_conexion, text="
50                                     Conectar",
51                                     command=self.conectar,
52                                     bg='#4CAF50', fg='white')
53         self.btn_conectar.pack(side=tk.LEFT, padx=5)
54
55         self.label_estado = tk.Label(frame_conexion, text="
56                                     Desconectado",
57                                     bg='#2b2b2b', fg='red')
```



```
55     self.label_estado.pack(side=tk.LEFT, padx=20)
56
57     frame_colores = tk.Frame(self.root, bg='#2b2b2b')
58     frame_colores.pack(pady=30)
59
60     # Rojo
61     frame_r = tk.Frame(frame_colores, bg='#2b2b2b')
62     frame_r.pack(side=tk.LEFT, padx=20)
63
64     self.label_r = tk.Label(frame_r, text="ROJO", font=('Arial',
65         14, 'bold'),
66         bg='#2b2b2b', fg='#ff4444')
67     self.label_r.pack()
68
69     self.canvas_r = tk.Canvas(frame_r, width=100, height=100,
70         bg='#1a1a1a')
71     self.canvas_r.pack()
72     self.rect_r = self.canvas_r.create_rectangle(0, 0, 100,
73         100, fill='#000000')
74
75     self.valor_r = tk.Label(frame_r, text="0%", font=('Arial',
76         16, 'bold'),
77         bg='#2b2b2b', fg='white')
78     self.valor_r.pack()
79
80     # Verde
81     frame_g = tk.Frame(frame_colores, bg='#2b2b2b')
82     frame_g.pack(side=tk.LEFT, padx=20)
83
84     self.label_g = tk.Label(frame_g, text="VERDE", font=('Arial',
85         14, 'bold'),
86         bg='#2b2b2b', fg='#44ff44')
87     self.label_g.pack()
88
89     self.canvas_g = tk.Canvas(frame_g, width=100, height=100,
90         bg='#1a1a1a')
91     self.canvas_g.pack()
92     self.rect_g = self.canvas_g.create_rectangle(0, 0, 100,
93         100, fill='#000000')
94
95     self.valor_g = tk.Label(frame_g, text="0%", font=('Arial',
96         16, 'bold'),
97         bg='#2b2b2b', fg='white')
98     self.valor_g.pack()
99
100    # Azul
101    frame_b = tk.Frame(frame_colores, bg='#2b2b2b')
102    frame_b.pack(side=tk.LEFT, padx=20)
103
104    self.label_b = tk.Label(frame_b, text="AZUL", font=('Arial',
105        14, 'bold'),
```

```
97         bg='#2b2b2b', fg='#4444ff')
98     self.label_b.pack()
99
100     self.canvas_b = tk.Canvas(frame_b, width=100, height=100,
101                               bg='#1a1a1a')
102     self.canvas_b.pack()
103     self.rect_b = self.canvas_b.create_rectangle(0, 0, 100,
104                                                  100, fill='#000000')
105
106     self.valor_b = tk.Label(frame_b, text="0%", font=('Arial',
107                                                    16, 'bold'),
108                             bg='#2b2b2b', fg='white')
109     self.valor_b.pack()
110
111     # Color resultante
112     frame_resultado = tk.Frame(self.root, bg='#2b2b2b')
113     frame_resultado.pack(pady=30)
114
115     tk.Label(frame_resultado, text="Color resultante:",
116             font=('Arial', 12), bg='#2b2b2b', fg='white').pack()
117
118     self.canvas_resultado = tk.Canvas(frame_resultado, width
119                                       =200, height=50,
120                                       bg='#000000',
121                                       highlightthickness=2,
122                                       highlightbackground='white')
123
124     self.canvas_resultado.pack()
125
126     def conectar(self):
127         try:
128             self.puerto = self.entry_puerto.get()
129             self.ser = serial.Serial(self.puerto, self.baudrate,
130                                     timeout=0.1)
131             self.serial_activo = True
132
133             self.label_estado.config(text="Conectado", fg='green')
134             self.btn_conectar.config(state='disabled')
135
136             self.hilo = threading.Thread(target=self.leer_datos,
137                                         daemon=True)
138             self.hilo.start()
139
140         except Exception as e:
141             self.label_estado.config(text=f"Error: {e}", fg='red')
142
143     def leer_datos(self):
144         while self.serial_activo:
145             try:
146                 if self.ser and self.ser.in_waiting:
```

```
139         linea = self.ser.readline().decode('utf-8').  
140             strip()  
141         match = self.patron.search(linea)  
142         if match:  
143             r = int(match.group(1))  
144             g = int(match.group(2))  
145             b = int(match.group(3))  
146             self.root.after(0, self.actualizar_valores,  
147                             r, g, b)  
148         except:  
149             pass  
150         time.sleep(0.05)  
151  
152     def actualizar_valores(self, r, g, b):  
153         self.r_val = r  
154         self.g_val = g  
155         self.b_val = b  
156  
157         self.valor_r.config(text=f"{r}%")  
158         self.valor_g.config(text=f"{g}%")  
159         self.valor_b.config(text=f"{b}%")  
160  
161         r_hex = int(r * 2.55)  
162         g_hex = int(g * 2.55)  
163         b_hex = int(b * 2.55)  
164  
165         color_r = f'#{r_hex:02x}0000'  
166         color_g = f'#00{g_hex:02x}00'  
167         color_b = f'#0000{b_hex:02x}'  
168         color_rgb = f'#{r_hex:02x}{g_hex:02x}{b_hex:02x}'  
169  
170         self.canvas_r.itemconfig(self.rect_r, fill=color_r)  
171         self.canvas_g.itemconfig(self.rect_g, fill=color_g)  
172         self.canvas_b.itemconfig(self.rect_b, fill=color_b)  
173         self.canvas_resultado.configure(bg=color_rgb)  
174  
175     def iniciar(self):  
176         self.root.mainloop()  
177  
178 if __name__ == "__main__":  
179     import serial.tools.list_ports  
180     puertos = [p.device for p in serial.tools.list_ports.comports()  
181                 ]  
182     puerto = puertos[0] if puertos else 'COM3'  
183     app = ControlRGB(puerto=puerto)  
184     app.iniciar()
```

6. Preguntas de Repaso

1. ¿Qué es el ciclo de trabajo en un PWM?

Es el porcentaje de tiempo que la señal permanece en nivel alto. Se calcula como:
$$\text{Ciclo} = \frac{t_{on}}{T} \times 100 \%$$

2. ¿Qué significa que el ciclo de trabajo del PWM sea de 40 %?

Significa que durante el 40 % del período la señal está en alto (3.3V) y el 60 % en bajo (0V). El LED brillará al 40 % de su intensidad máxima.

3. ¿Cómo se genera una señal PWM en un microcontrolador?

Se usa un temporizador que cuenta hasta un valor máximo (wrap). Un registro de comparación determina cuándo cambiar la salida.

4. ¿Qué efecto tiene variar la frecuencia de la señal PWM en la iluminación del LED RGB?

Frecuencias bajas (<100 Hz) causan parpadeo visible. Frecuencias altas (1-10 kHz) dan luz continua.

5. ¿Cuál es la relación entre la resolución del ADC y la precisión del control PWM?

ADC de 12 bits da 4096 valores, PWM de 8 bits tiene 256 niveles. La precisión está limitada por el PWM.

6. ¿Cómo se pueden mejorar las transiciones de color en el LED RGB?

Usando filtros, interpolación lineal, mayor frecuencia de actualización o PWM de mayor resolución.

7. ¿Qué aplicaciones prácticas tiene el uso de PWM en sistemas de control?

Control de motores DC, regulación de brillo, control de temperatura, carga de baterías, servomotores.

8. ¿Cuál es la diferencia entre PWM y PAM?

PWM varía el ancho del pulso con amplitud constante. PAM varía la amplitud del pulso con ancho constante.

7. Conclusiones

7.1. Aprendizajes de la práctica

- Se comprendió el funcionamiento del PWM y su aplicación en control de intensidad lumínica
- Se implementó la lectura de tres potenciómetros usando el ADC de 12 bits
- Se logró mezclar colores en un LED RGB controlando independientemente cada canal
- Se desarrolló una interfaz gráfica en Python para visualizar los valores en tiempo real
- Se aprendió a configurar múltiples canales PWM en la Raspberry Pi Pico W

7.2. Posibles mejoras

- Implementar modos automáticos (barrido de colores, arcoíris)
- Agregar memoria para guardar colores favoritos
- Control mediante interfaz web desde el celular
- Usar PWM de mayor resolución (16 bits) para transiciones más suaves
- Implementar curvas de gamma para percepción lineal del brillo

8. Referencias

1. Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*.
2. Raspberry Pi Foundation. (2024). *Pico SDK Documentation - PWM and ADC*.
3. RP2040 Datasheet. (2024). *Chapter 4: PWM and ADC*.